

Projeto e Análise de Algoritmos

Árvore Geradora Mínima

Thiago Pinheiro de Araújo

EMAp/FGV

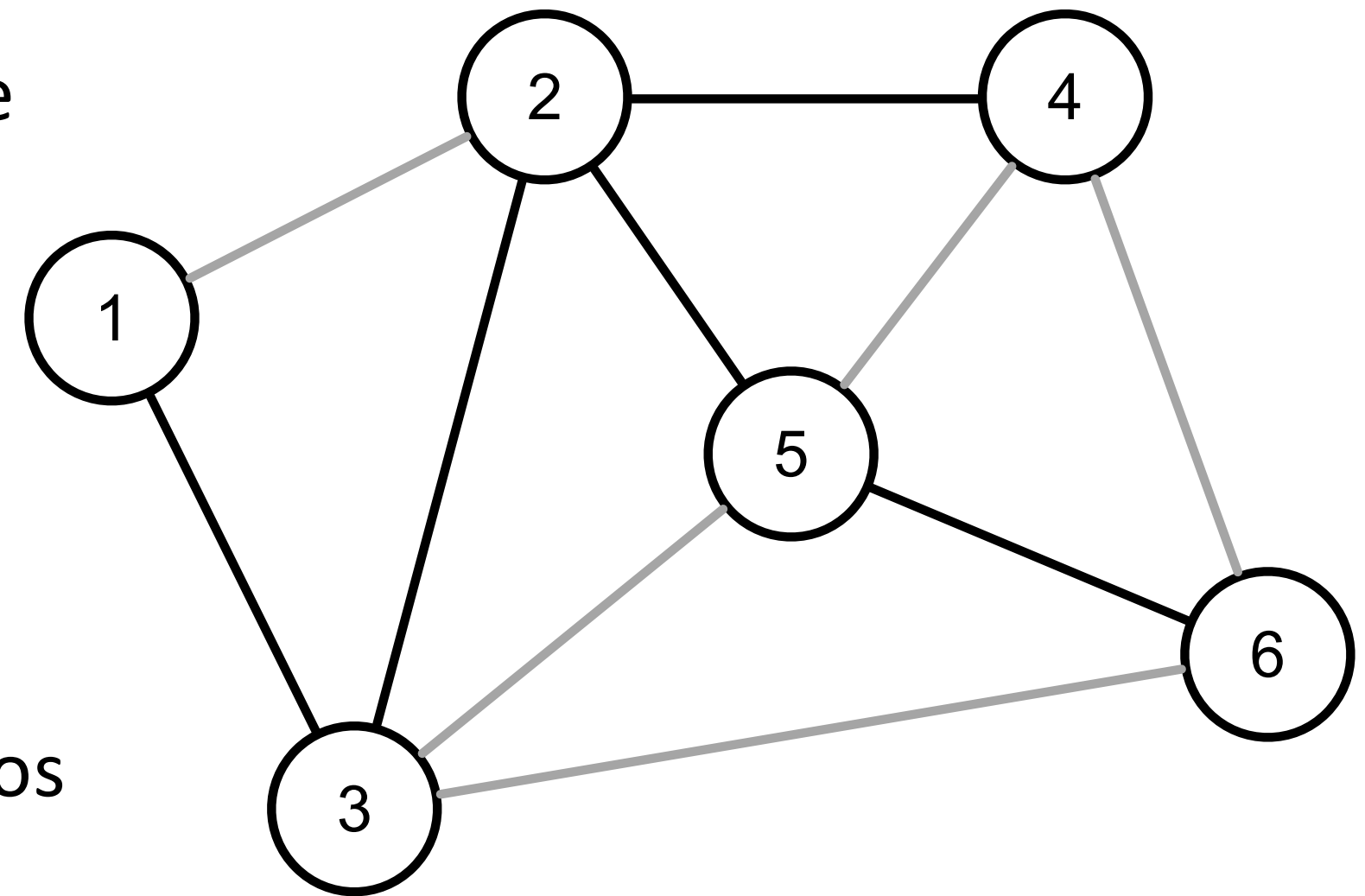
2025.2

Apresentar o conceito de **árvore geradora mínima** em um grafo e algoritmos para encontrá-la. Para cada algoritmo analisaremos a sua complexidade e apresentaremos um exemplo de implementação.

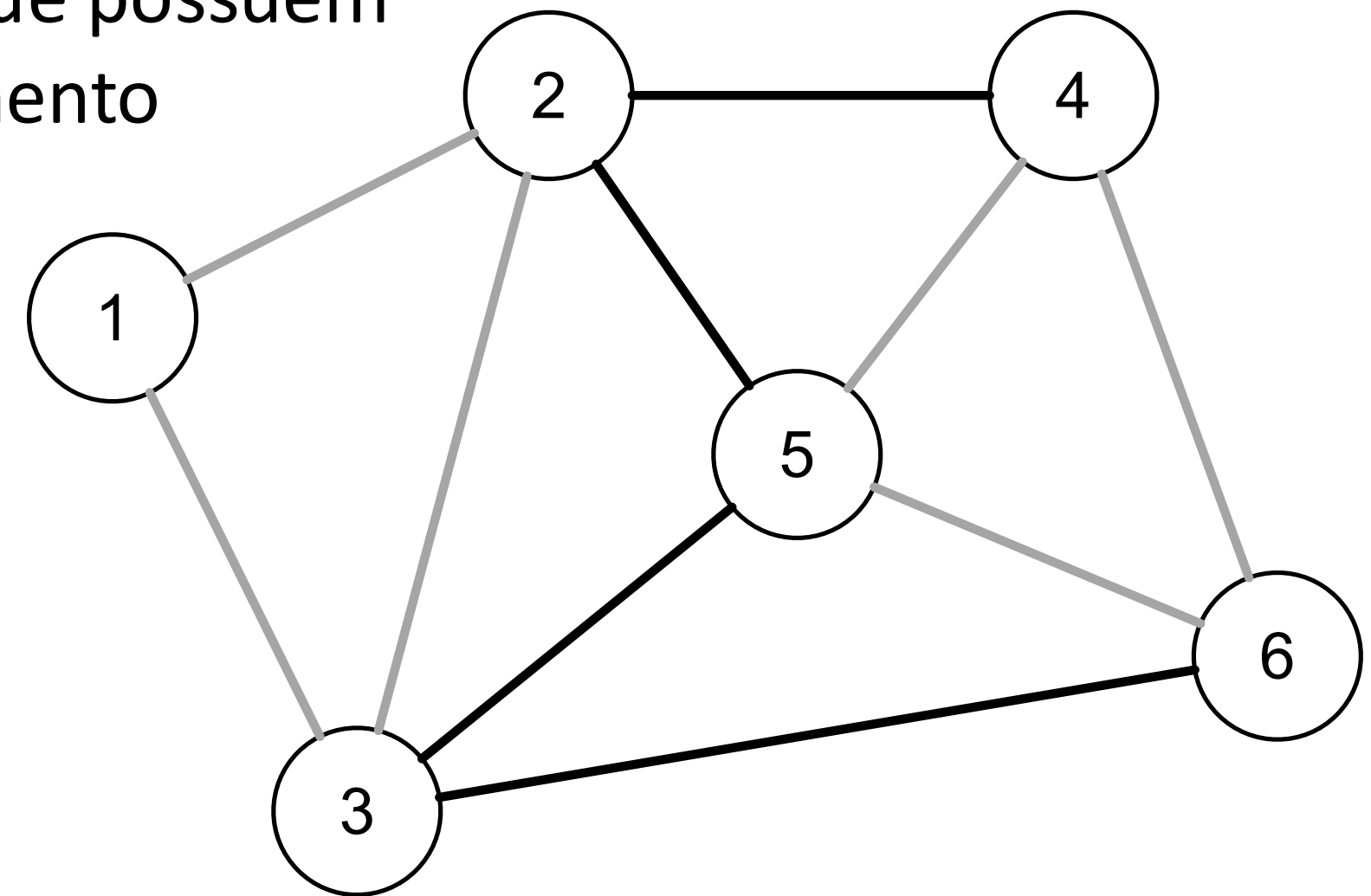
Árvore Geradora Mínima

Introdução

- Uma **árvore geradora** de um grafo $G = (V, E)$ é um subgrafo T que não possua ciclos e que contenha todos os vértices de G .
- Se um grafo G possui uma árvore geradora ele é conexo, visto que toda árvore é conexa.
- Toda árvore geradora de um grafo possui exatamente $V - 1$ arestas.
- Os exemplos a seguir irão assumir que os grafos são conexos e não-dirigidos.



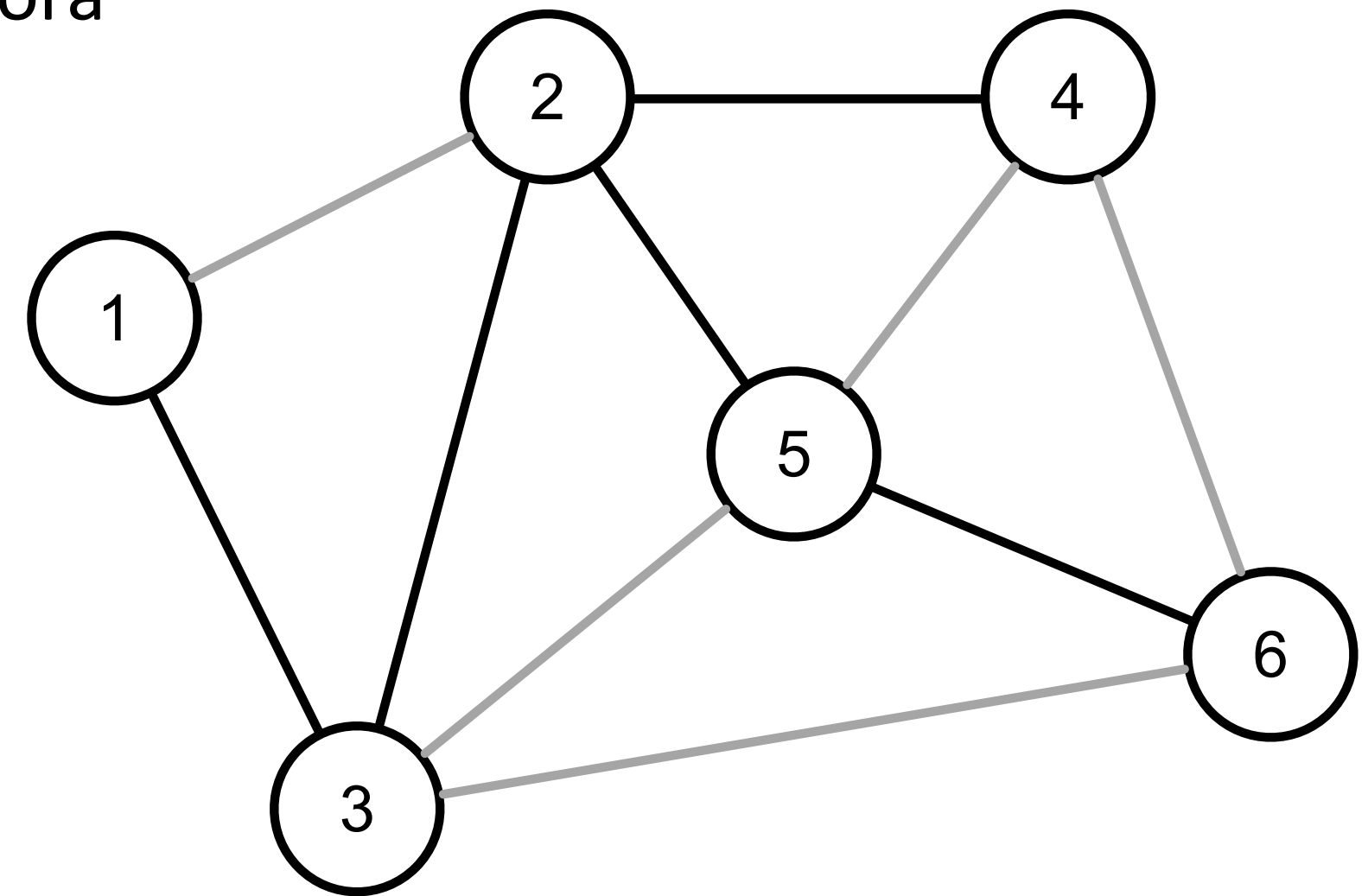
- Um **corte** é um conjunto de arestas que conecta duas partes de um grafo.
- Dado um conjunto A de vértices, as arestas que possuem uma ponta em A e a outra ponta no complemento de A representam um corte.
- A e $\bar{A}$ são as margens do corte.



Árvore Geradora Mínima

Introdução

- A manipulação de árvores geradoras possui duas operações básicas.
- A **adição** de uma aresta em uma árvore geradora cria um ciclo.
- A **remoção** de uma aresta em uma árvore geradora cria um corte.
- A partir dessas operações temos duas propriedades de substituição de arestas.



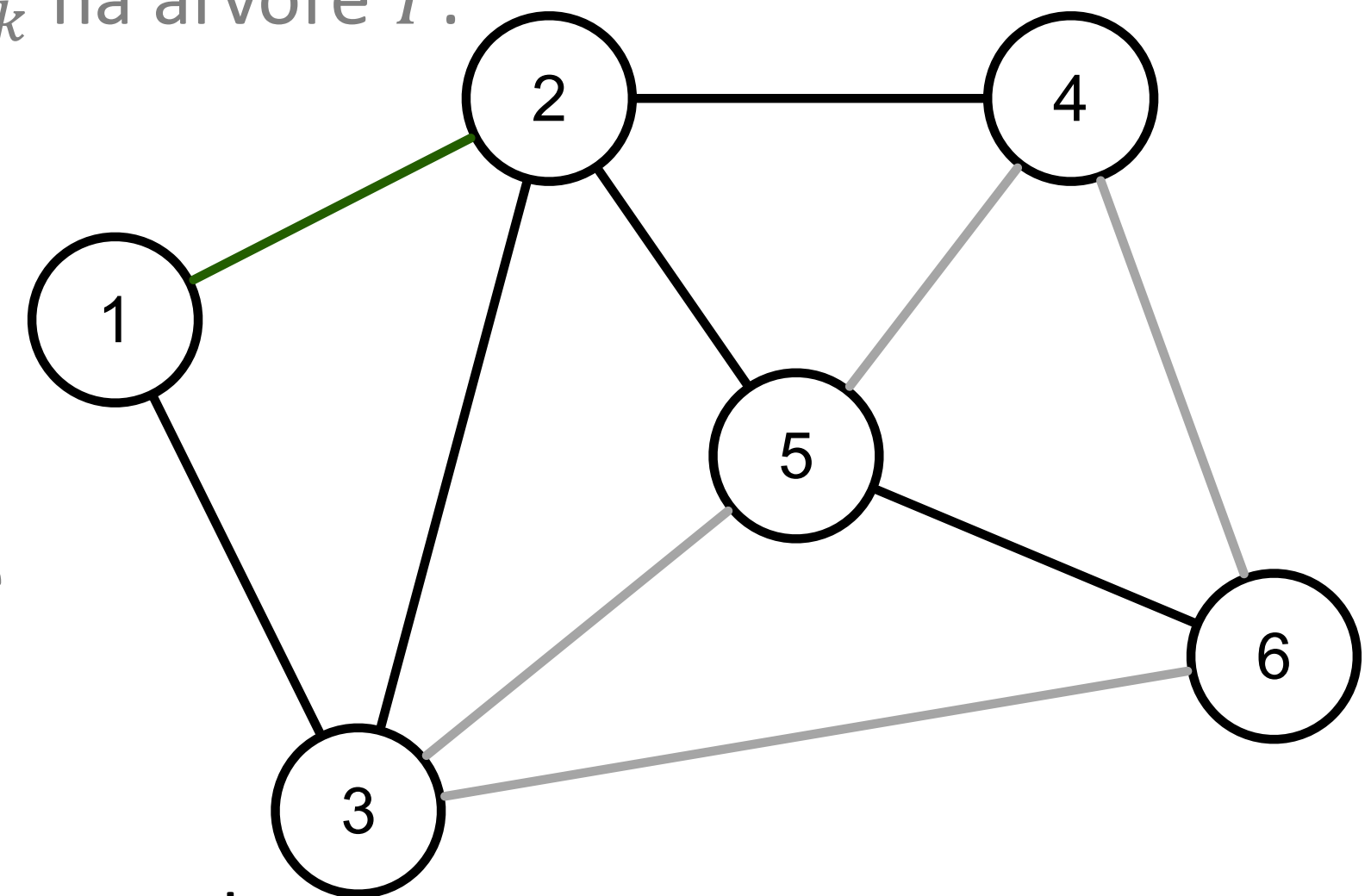
Árvore Geradora Mínima

Introdução

- Seja T uma árvore geradora de um grafo $G = (V, E)$, e e_k uma aresta de G :
 - $T + e_k$ é o grafo gerado pela adição de e_k na árvore T .
 - $T - e_k$ é o grafo gerado pela remoção de e_k na árvore T .

- A **propriedade dos ciclos** consiste em:
 - Se $e_i \notin T$, o grafo $T + e_i$ possui um único ciclo C .
 - Se $e_j \in C$, o grafo $T + e_i - e_j$ é uma árvore geradora.

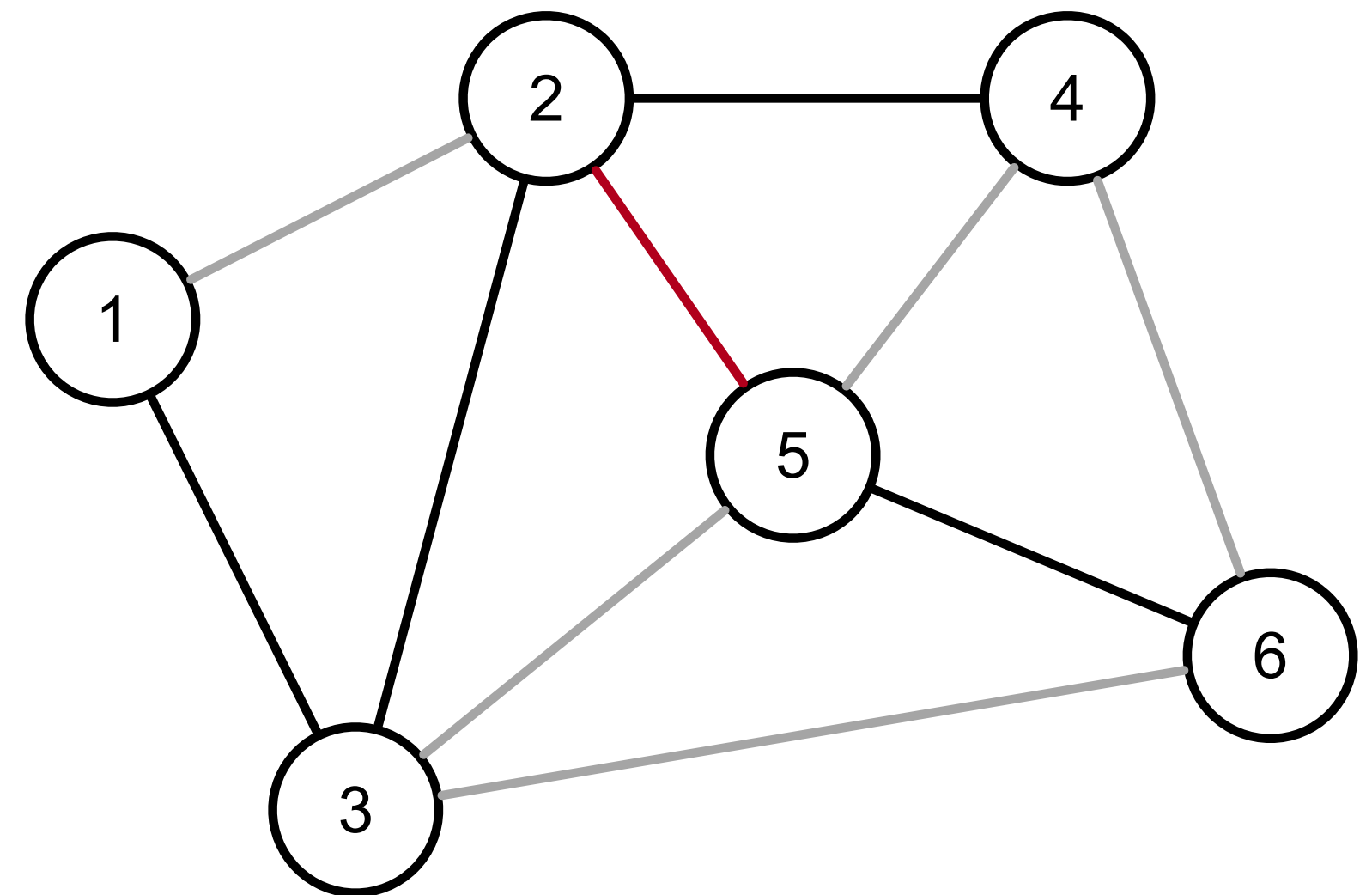
- O ciclo em $T + e_k$ é considerado o ciclo fundamental de e relativo à T .



Árvore Geradora Mínima

Introdução

- Seja T uma árvore geradora de um grafo $G = (V, E)$, e e_k uma aresta de T :
 - $T - e_k$ produz uma floresta com duas componentes conexas de G .
- A **propriedade dos cortes** consiste em:
 - Dado $e_i \in T$ e $e_j \in \text{corte}(T - e_i)$.
 - $T - e_i + e_j$ é uma árvore geradora.
- O corte gerado por $T - e_k$ é considerado o corte fundamental de e_k relativo à T .



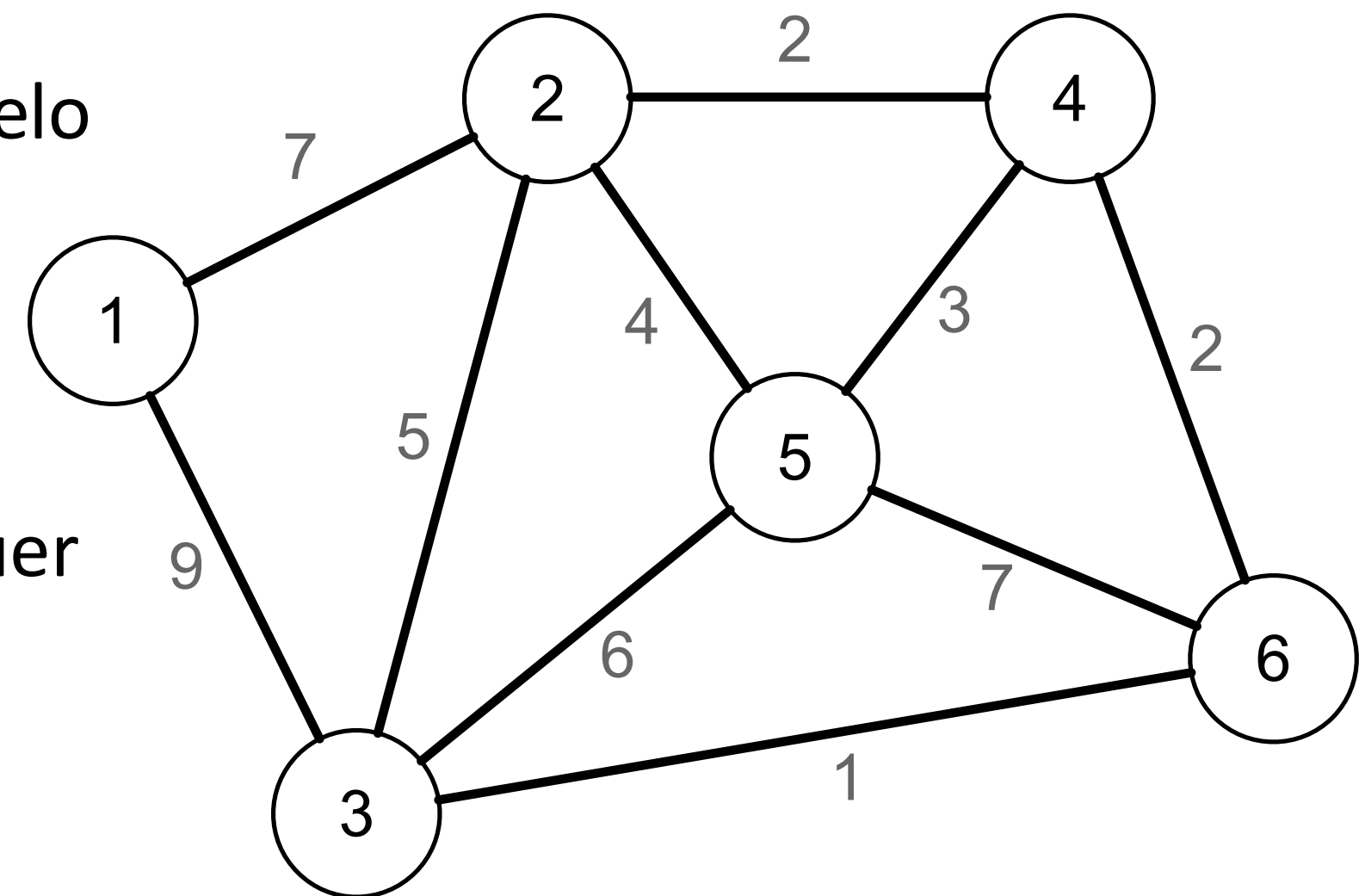
Árvore Geradora Mínima

Introdução

- Seja $G = (V, E)$ um grafo não-dirigido com custo nas arestas
 - O custo pode assumir valores positivos e negativos.

- O custo de um subgrafo H de G é calculado pelo somatório do custos das arestas de H .

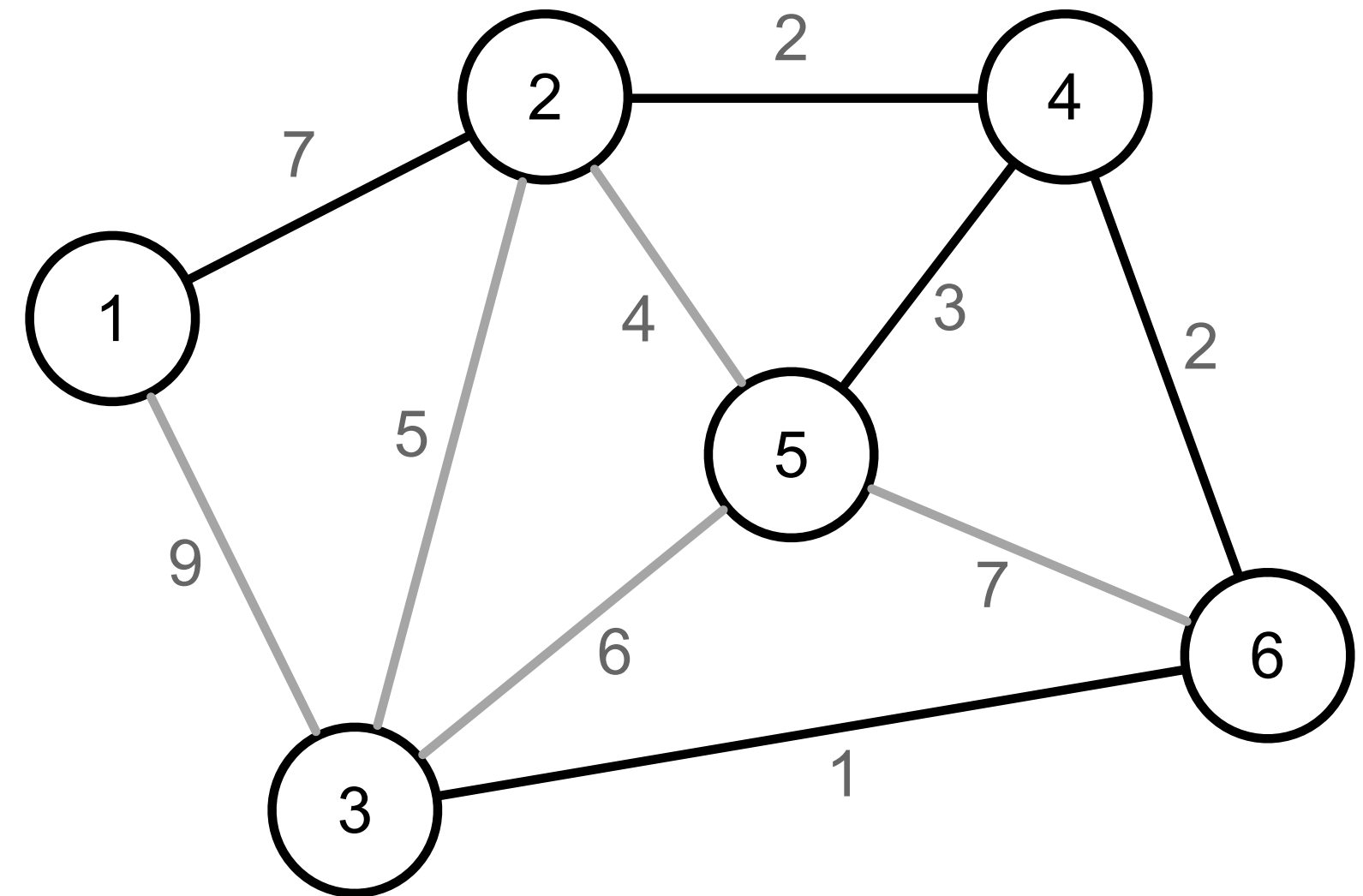
- Uma **árvore geradora mínima** (*Minimum Spanning Tree – MST*) de um grafo G é qualquer árvore geradora cujo custo seja mínimo.



Árvore Geradora Mínima

Introdução

- **Problema:** dado um grafo $G = (V, E)$ não-dirigido com custo nas arestas encontre uma árvore geradora mínima.
- Só existe solução se o grafo for conexo.
- Se todas as arestas apresentarem o mesmo custo toda árvore geradora será mínima.

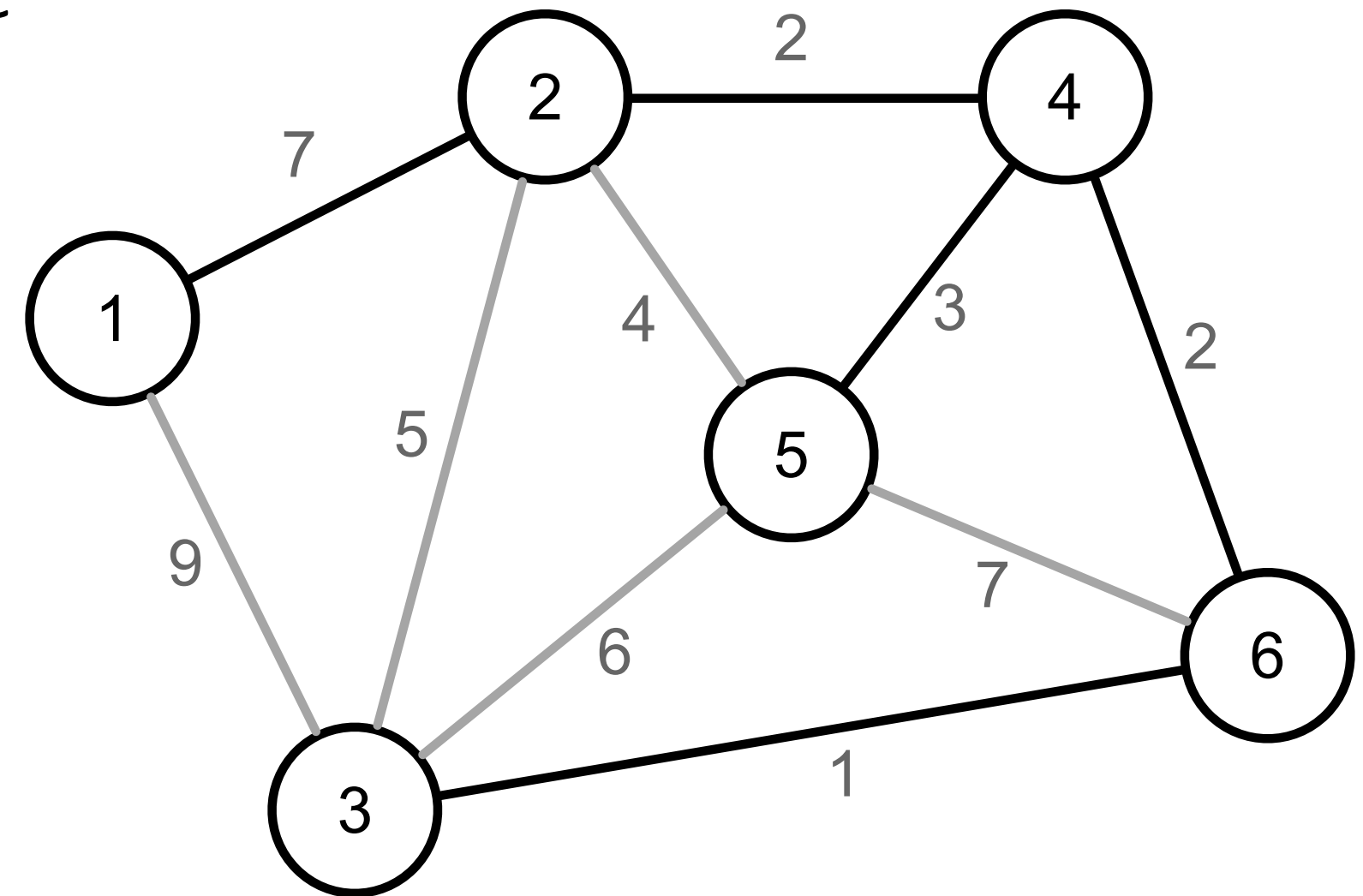


Algoritmo de Prim

Árvore Geradora Mínima

Algoritmo de Prim

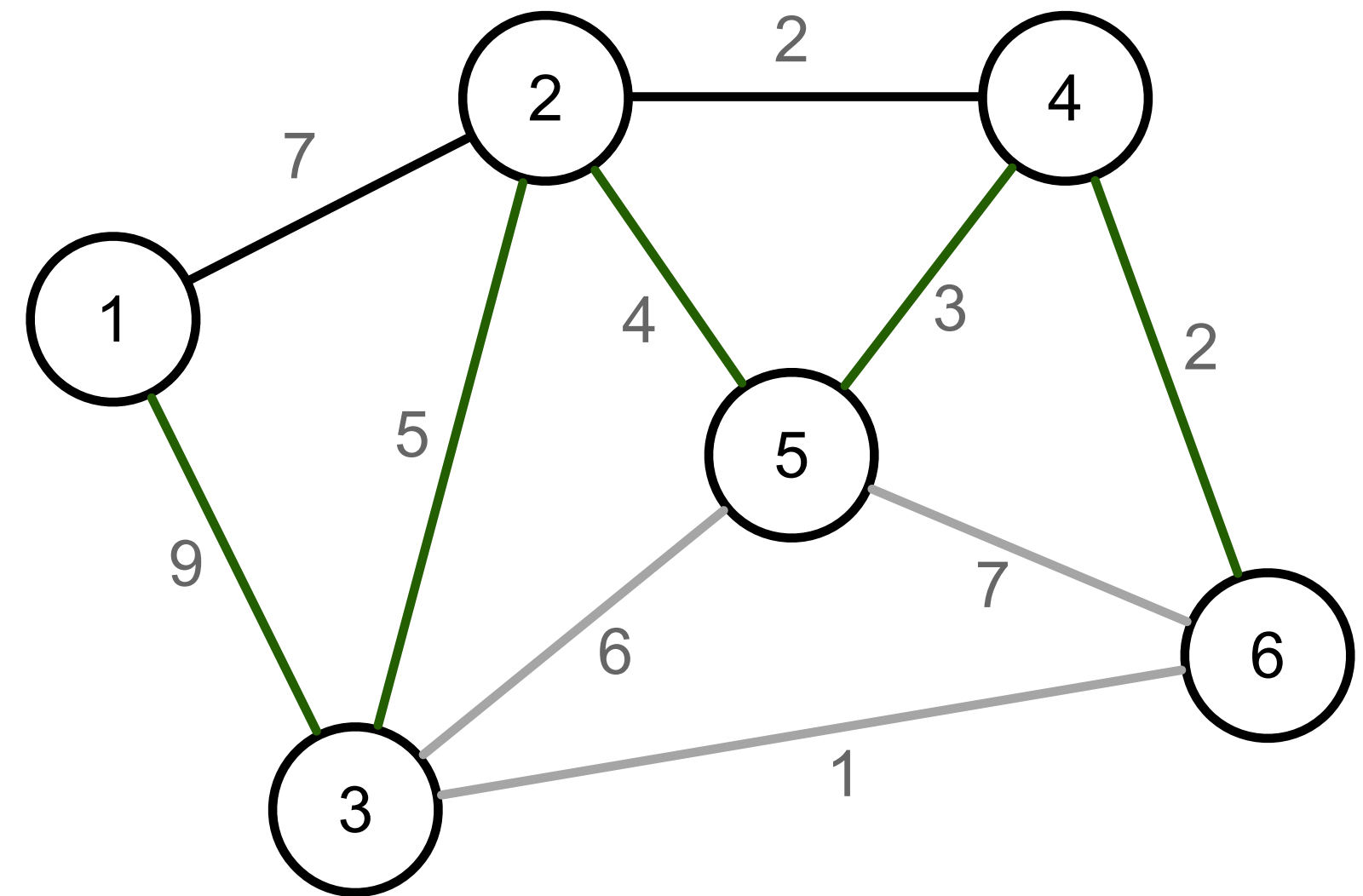
- O algoritmo de **Prim** foi publicado por Robert Prim (1957) e Dijkstra (1959).
- Esse algoritmo é capaz de encontrar a MST de um grafo $G = (V, E)$.
- A estratégia desse algoritmo consiste em crescer uma árvore a partir de um vértice v_0 arbitrário até que ela se torne geradora.



Árvore Geradora Mínima

Algoritmo de Prim

- Dada uma árvore T de G , considere a franja de T como o corte cuja margem é composta pelos vértices em T .
- Ideia geral do algoritmo:
 - Escolha um vértice para ser a raiz da árvore T .
 - Enquanto a franja de T não estiver vazia:
 - Escolha uma aresta $e = (v_i, v_j)$ da franja com custo mínimo.
 - Considerando que v_i é o vértice em T .
 - Insira e e v_j em T .



Árvore Geradora Mínima

Algoritmo de Prim

- **Exercício:** proponha uma implementação para o algoritmo.

```
void mstPrimSlow(vertex * parent) {
    for (vertex v=0; v < m_numVertices; v++) { parent[v] = -1; }
    parent[0] = 0;
    while (true) {
        int min = INT_MAX;
        vertex treeV, newV = -1;
        for (vertex v1=0; v1 < m_numVertices; v1++) {
            if (parent[v1] == -1) { continue; }
            EdgeNode * edge = m_edges[v1];
            while(edge) {
                vertex v2 = edge->otherVertex();
                int cost = edge->cost();
                if (parent[v2] == -1 && cost < min) {
                    min = cost;
                    treeV = v1;
                    newV = v2;
                }
                edge = edge->next();
            }
        }
        if (min == INT_MAX) { break; }
        parent[newV] = treeV;
    }
}
```


Árvore Geradora Mínima

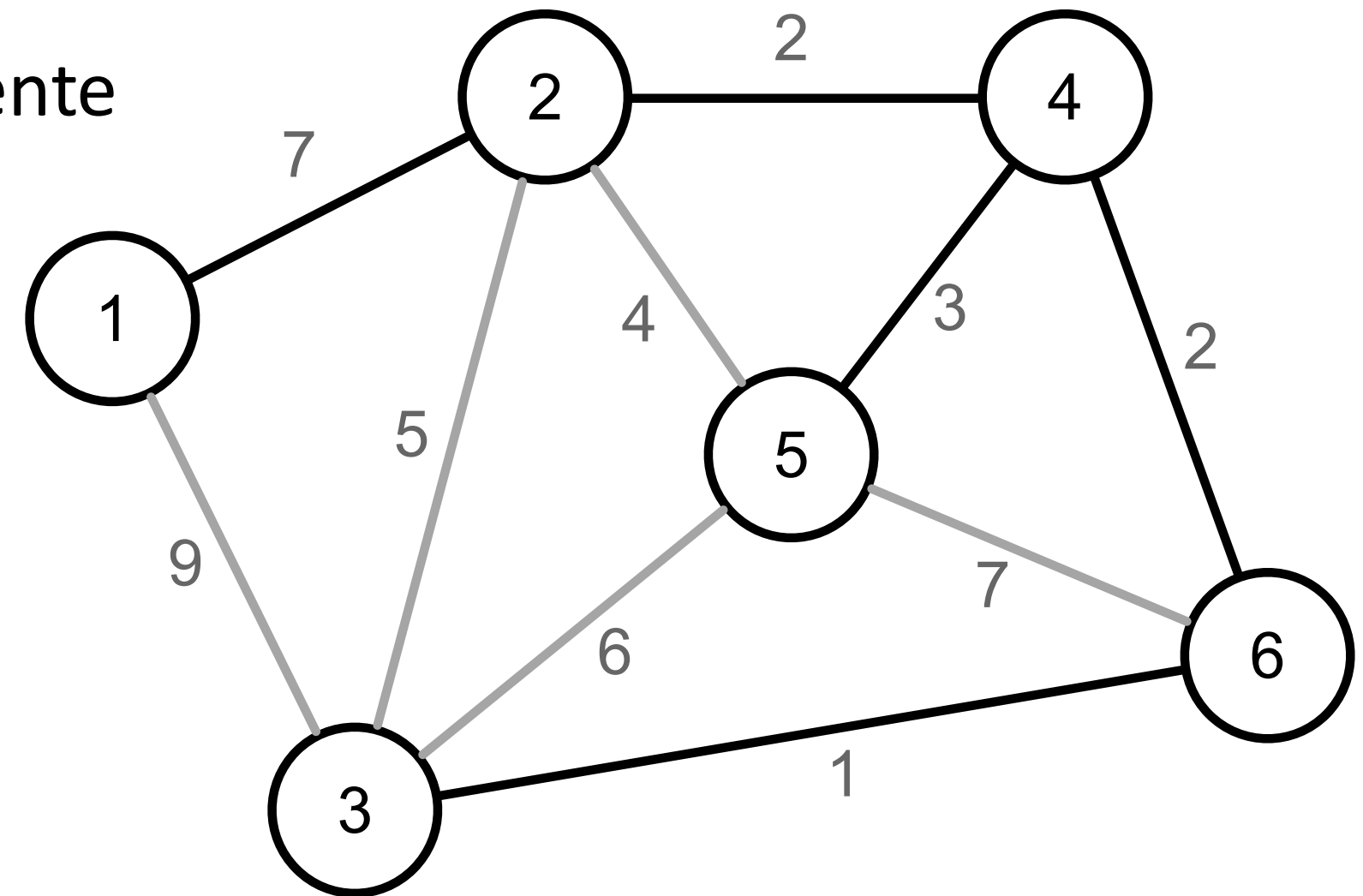
Algoritmo de Prim

- Analise da complexidade:
 - Cada iteração adiciona um novo vértice à árvore
 - Para cada vértice $v \in V$
 - Verifica-se todas as suas arestas – $g_s(v)$.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $O(VE)$
 - Se o grafo for denso, a complexidade será $O(V^3)$.

Árvore Geradora Mínima

Algoritmo de Prim

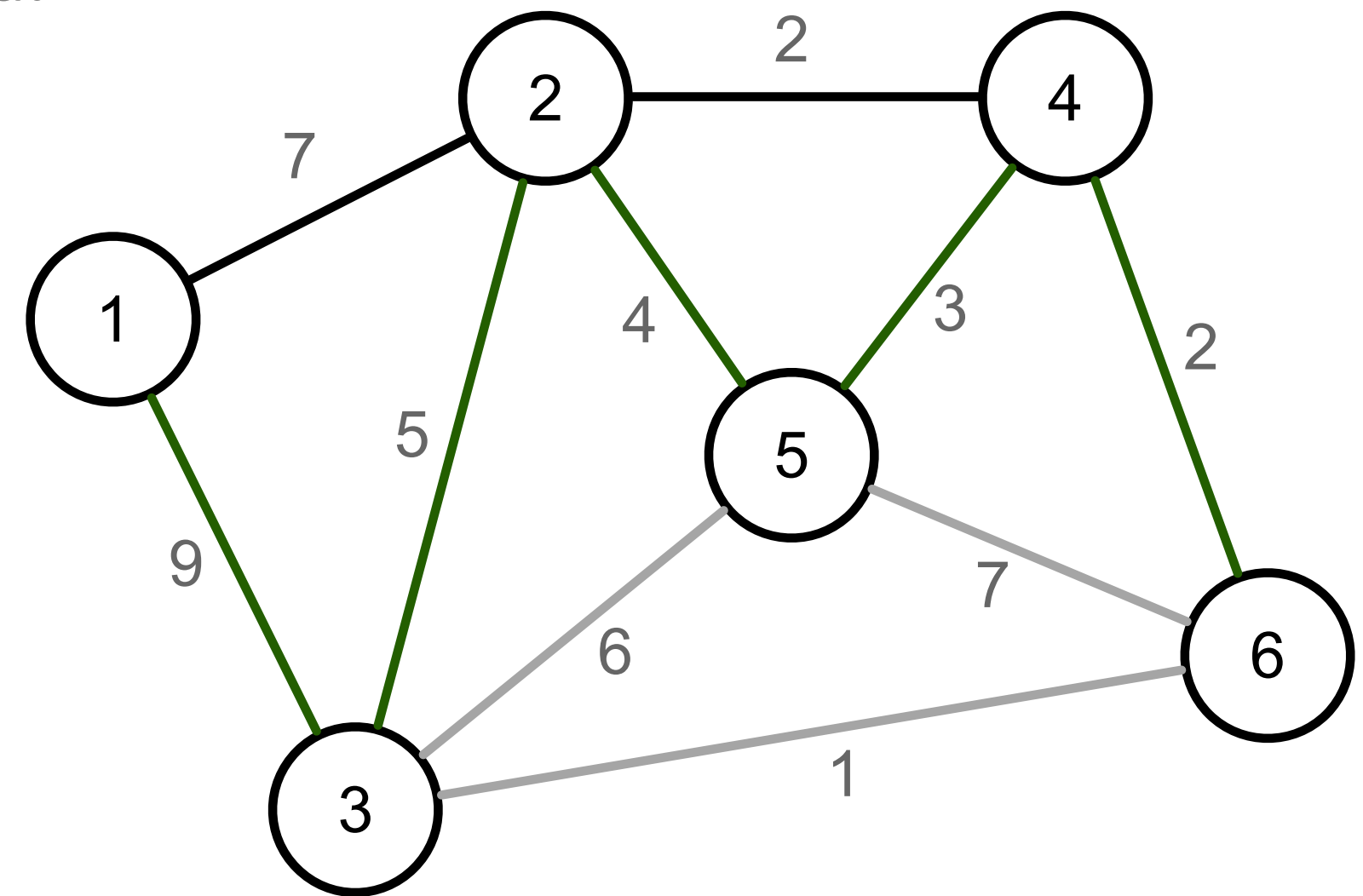
- A corretude do algoritmo pode ser avaliada através do **critério de minimalidade baseado em cortes**.
- Uma árvore geradora T é uma MST se e somente se cada aresta $e_k \in T$ apresentar o menor custo no corte fundamental de e_k relativo à T .



Árvore Geradora Mínima

Algoritmo de Prim

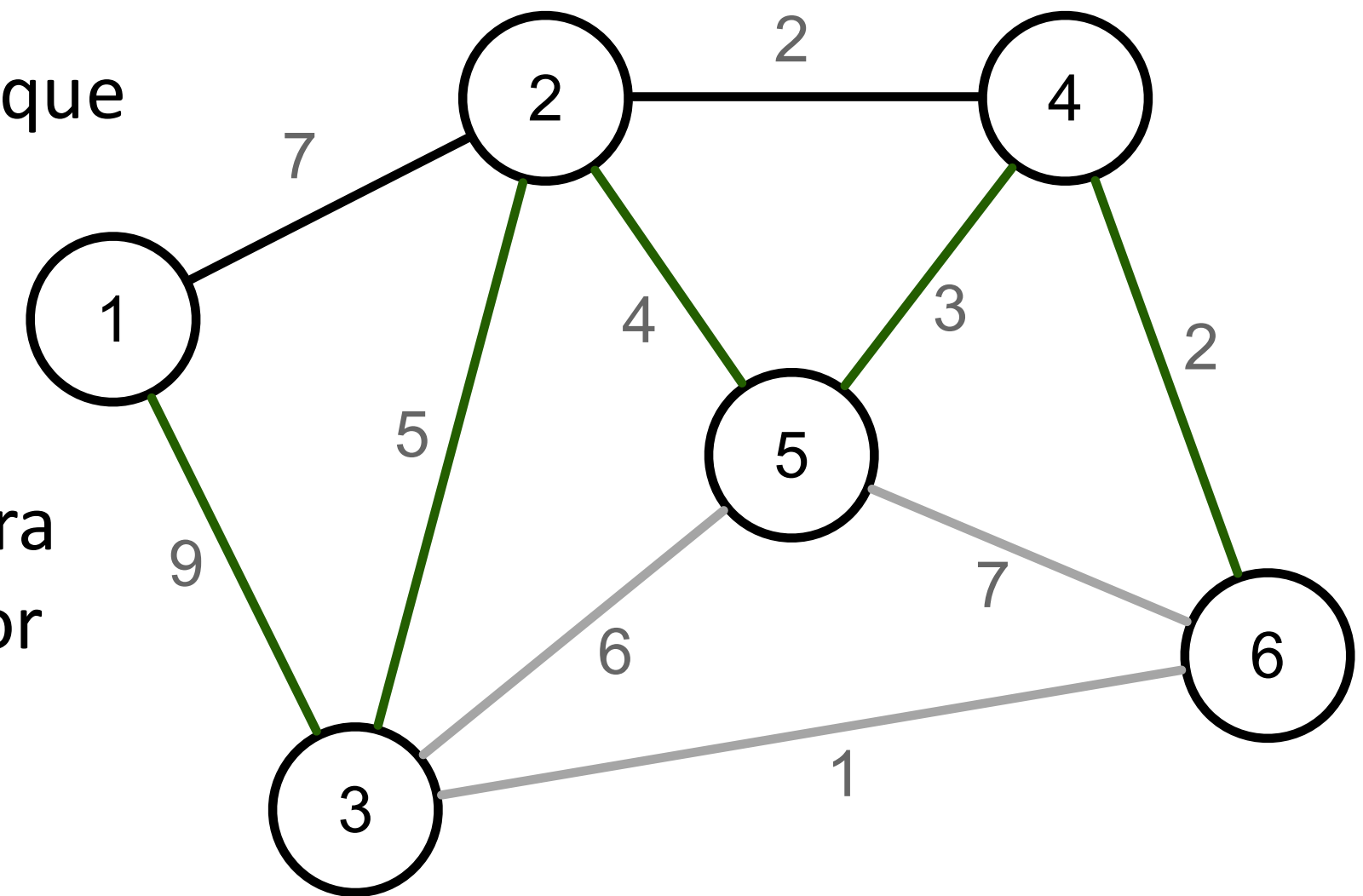
- A versão simplória do algoritmo é lenta por recalcular toda a franja à cada interação
 - Observe que as margens são modificadas gradualmente à medida que a árvore vai sendo construída.
- Podemos encontrar uma solução mais eficiente apenas atualizando a franja entre interações.



Árvore Geradora Mínima

Algoritmo de Prim

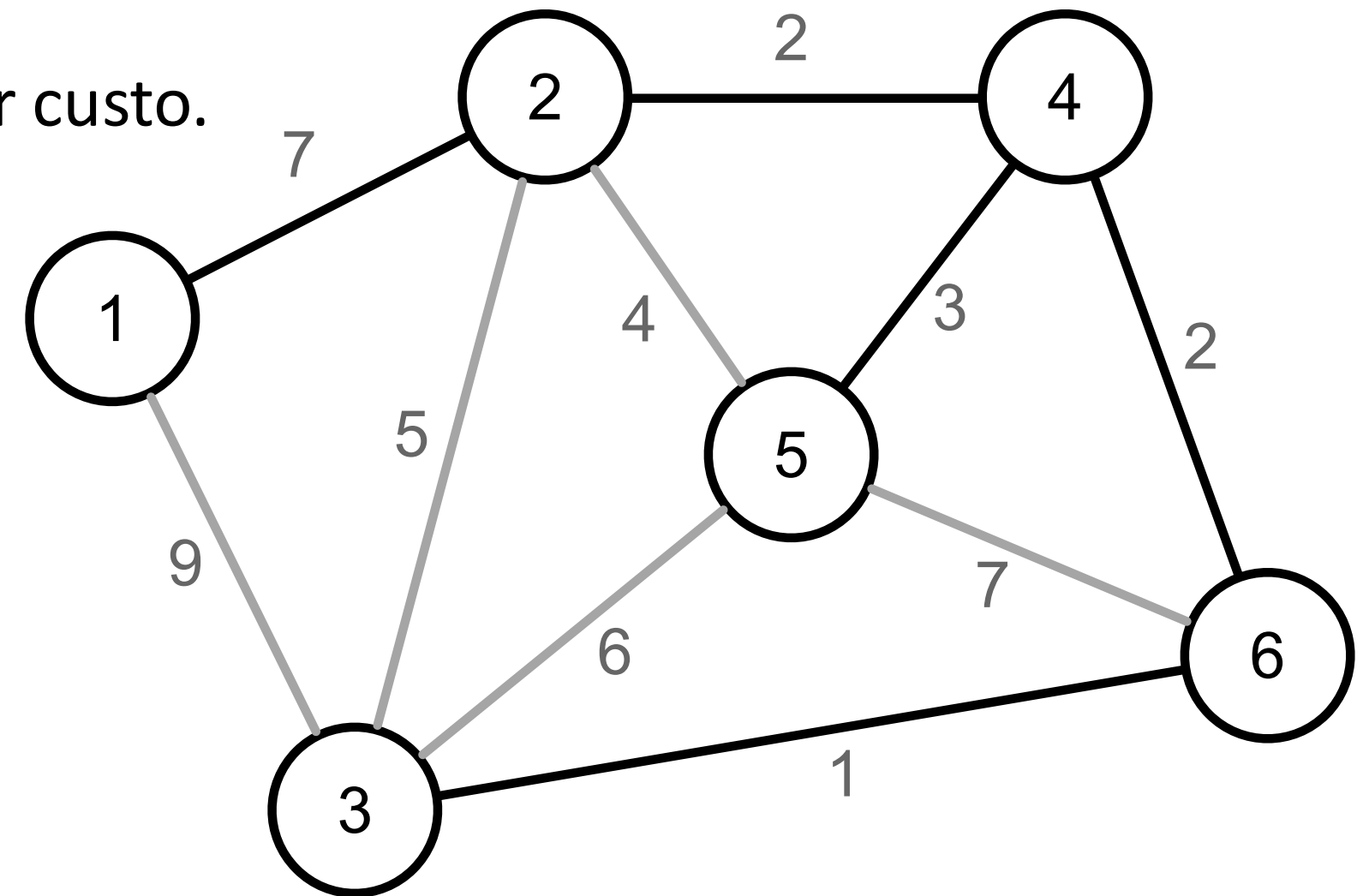
- Uma estratégia consiste em manter uma estrutura de dados com o custo de cada vértice que pode ser adicionado à T .
- Considere a fronteira como todos os vértices que não pertencem à T e podem ser acessados a partir da franja.
- O custo de adicionar um vértice v_k da fronteira à T é definido pelo custo da aresta com menor custo que incide em v_k e pertence à franja.



Árvore Geradora Mínima

Algoritmo de Prim

- Ideia geral do algoritmo:
 - Escolha um vértice para ser a raiz da árvore T .
 - Enquanto a franja de T não estiver vazia:
 - Procure o vértice v_k na fronteira com o menor custo.
 - Adicione o vértice v_k na árvore.
 - Atualize a fronteira adicionando os vértices acessíveis a partir de v_k que ainda não estão em T .



Árvore Geradora Mínima



Algoritmo de Prim

- Uma possível implementação desse algoritmo seria:

```
void initialize(vertex * parent, bool * inTree,
               int * vertexCost) const {
    for (vertex v=0; v < m_numVertices; v++) {
        parent[v] = -1;
        inTree[v] = false;
        vertexCost[v] = INT_MAX;
    }
    parent[0] = 0;
    inTree[0] = true;
    EdgeNode * edge = m_edges[0];
    while(edge) {
        vertex v2 = edge->otherVertex();
        parent[v2] = 0;
        vertexCost[v2] = edge->cost();
        edge = edge->next();
    }
}
```

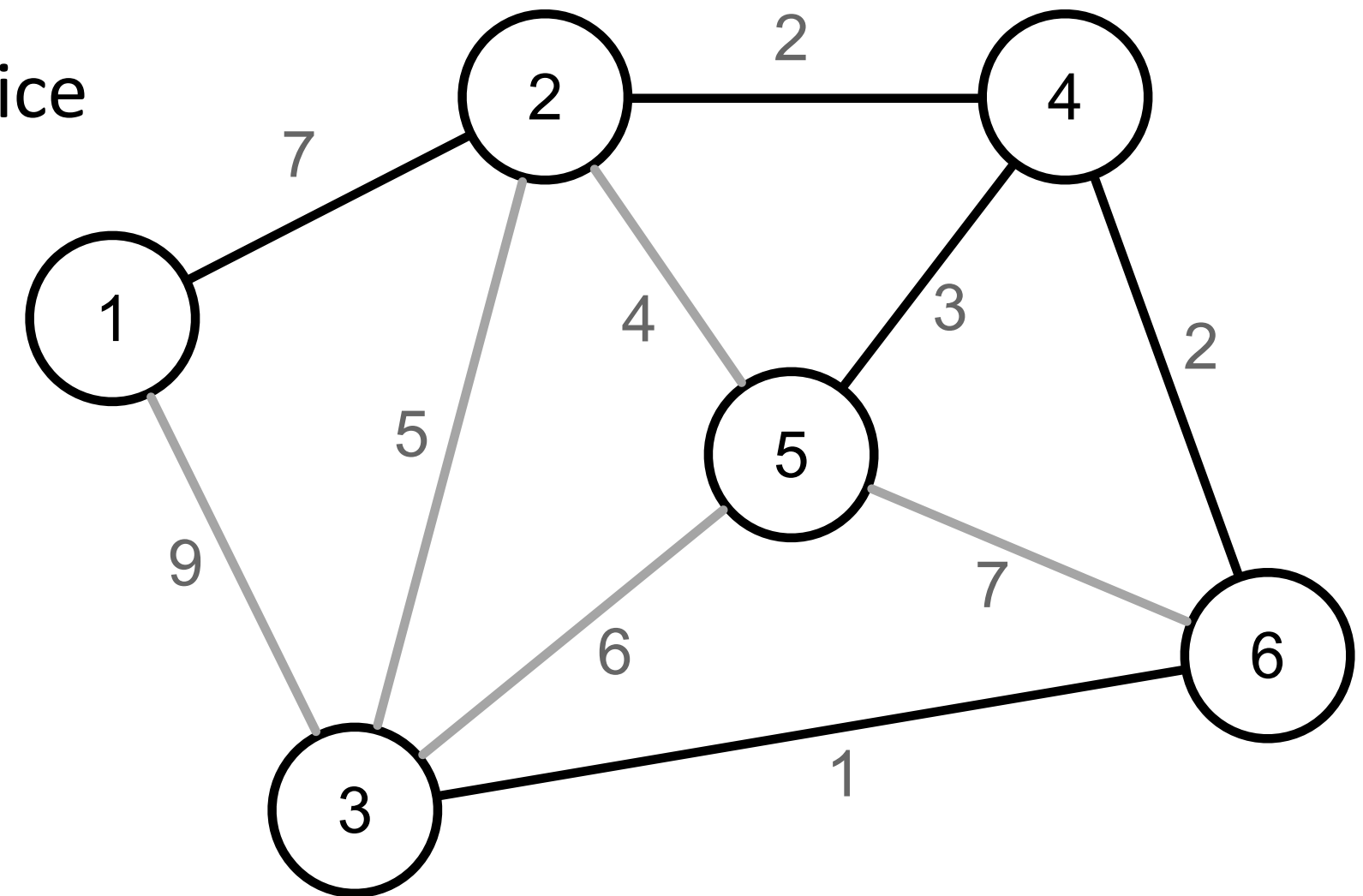
```
void mstPrimFastV1(vertex * parent) {
    bool inTree[m_numVertices];
    int vertexCost[m_numVertices];
    initialize(parent, inTree, vertexCost);
    while (true) {
        int minCost = INT_MAX;
        vertex v1;
        for (vertex v=0; v < m_numVertices; v++) {
            if (!inTree[v] && vertexCost[v] < minCost) {
                minCost = vertexCost[v];
                v1 = v;
            }
        }
        if (minCost == INT_MAX) {
            break;
        }
        inTree[v1] = true;
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            int cost = edge->cost();
            if (!inTree[v2] && cost < vertexCost[v2]) {
                vertexCost[v2] = cost;
                parent[v2] = v1;
            }
            edge = edge->next();
        }
    }
}
```

- Analise da complexidade:
 - Cada vértice é avaliado uma única vez.
 - Em cada avaliação:
 - Procura-se o vértice v_k na fronteira cujo custo de adição à árvore seja mínimo.
 - Atualiza a fronteira nas $g_s(vk)$ arestas.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $O(V^2 + E)$.
- Sendo $E < V^2$, temos que a complexidade é $O(V^2)$.

Árvore Geradora Mínima

Algoritmo de Prim

- Uma outra forma de implementar o algoritmo consiste em manter os vértices da fronteira em ordem crescente de custo.
- Dessa forma não é necessário procurar o vértice que apresenta o menor custo à cada iteração.
- A ordenação pode ser implementada através de um heap mínimo representando uma fila de prioridades.



Árvore Geradora Mínima

Algoritmo de Prim

- Uma possível implementação desse algoritmo seria:

```
void mstPrimFastV2(vertex * parent) {
    bool inTree[m_numVertices];
    int vertexCost[m_numVertices];
    initialize(parent, inTree, vertexCost);
    Heap heap; // Create the heap
    for (vertex v = 1; v < m_numVertices; v++) { heap.insert_or_update(vertexCost[v], v); }
    while (!heap.empty()) {
        vertex v1 = heap.top().second; // Min vertex
        heap.pop(); // Remove from heap
        if (vertexCost[v1] == INT_MAX) { break; }
        inTree[v1] = true;
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            int cost = edge->cost();
            if (!inTree[v2] && cost < vertexCost[v2]) {
                vertexCost[v2] = cost;
                parent[v2] = v1;
                heap.insert_or_update(vertexCost[v2], v2);
            }
            edge = edge->next();
        }
    }
}
```

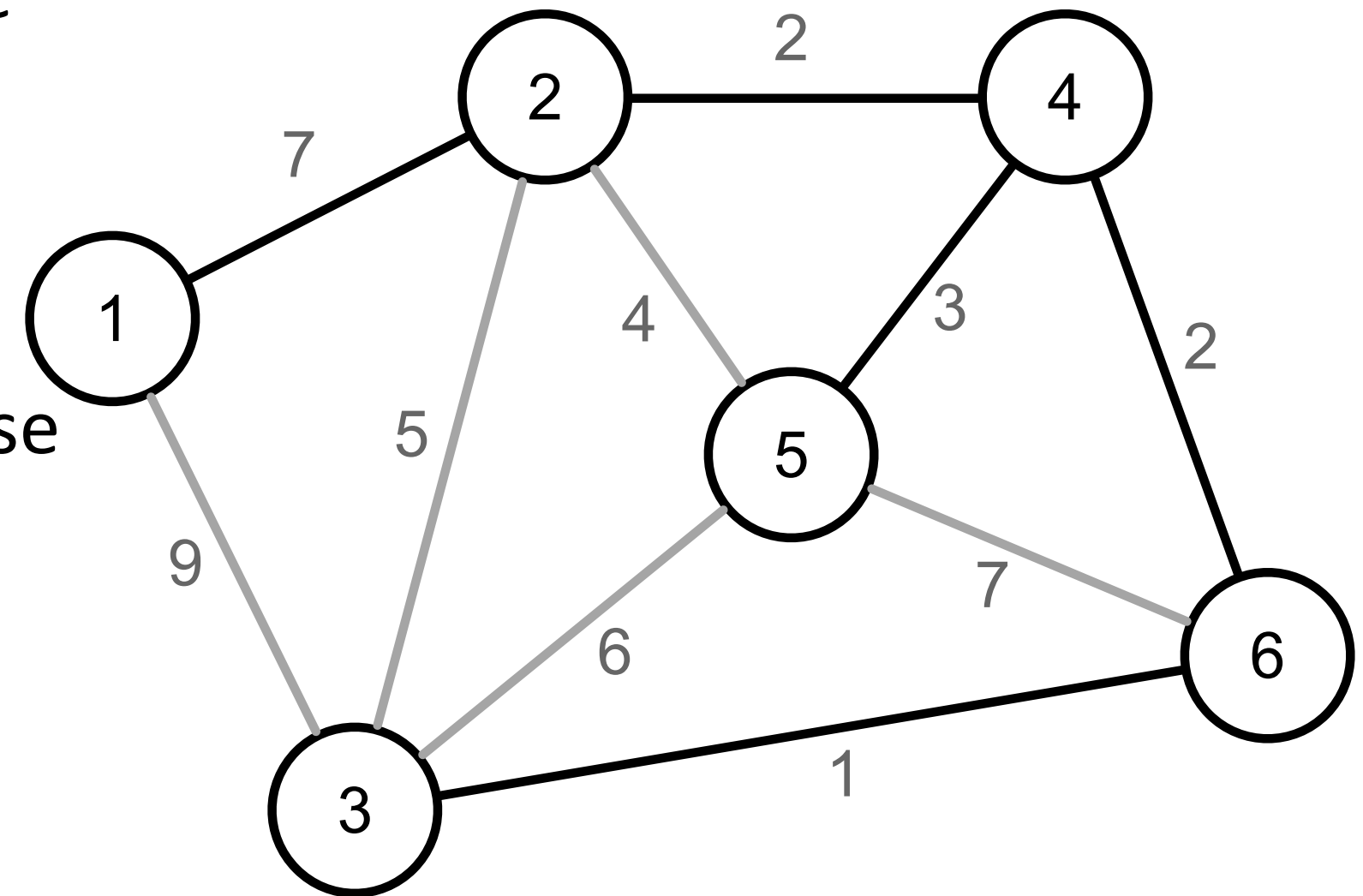
- Analise da complexidade:
 - Cada vértice é avaliado uma única vez.
 - Obtêm-se o vértice v_k na fronteira com menor custo em $O(\log(V))$.
 - Atualiza-se a fronteira em $g_s(vk)$ arestas.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $O((V + E) \log(V))$
 - Como assumimos que o grafo é conexo, temos a complexidade $O(E \log(V))$.
 - Se o grafo for denso, temos que a complexidade é $O(V^2 \log(V))$
 - Ou seja, apresenta um desempenho inferior à abordagem anterior.

Algoritmo de Kruskal

Árvore Geradora Mínima

Algoritmo de Kruskal

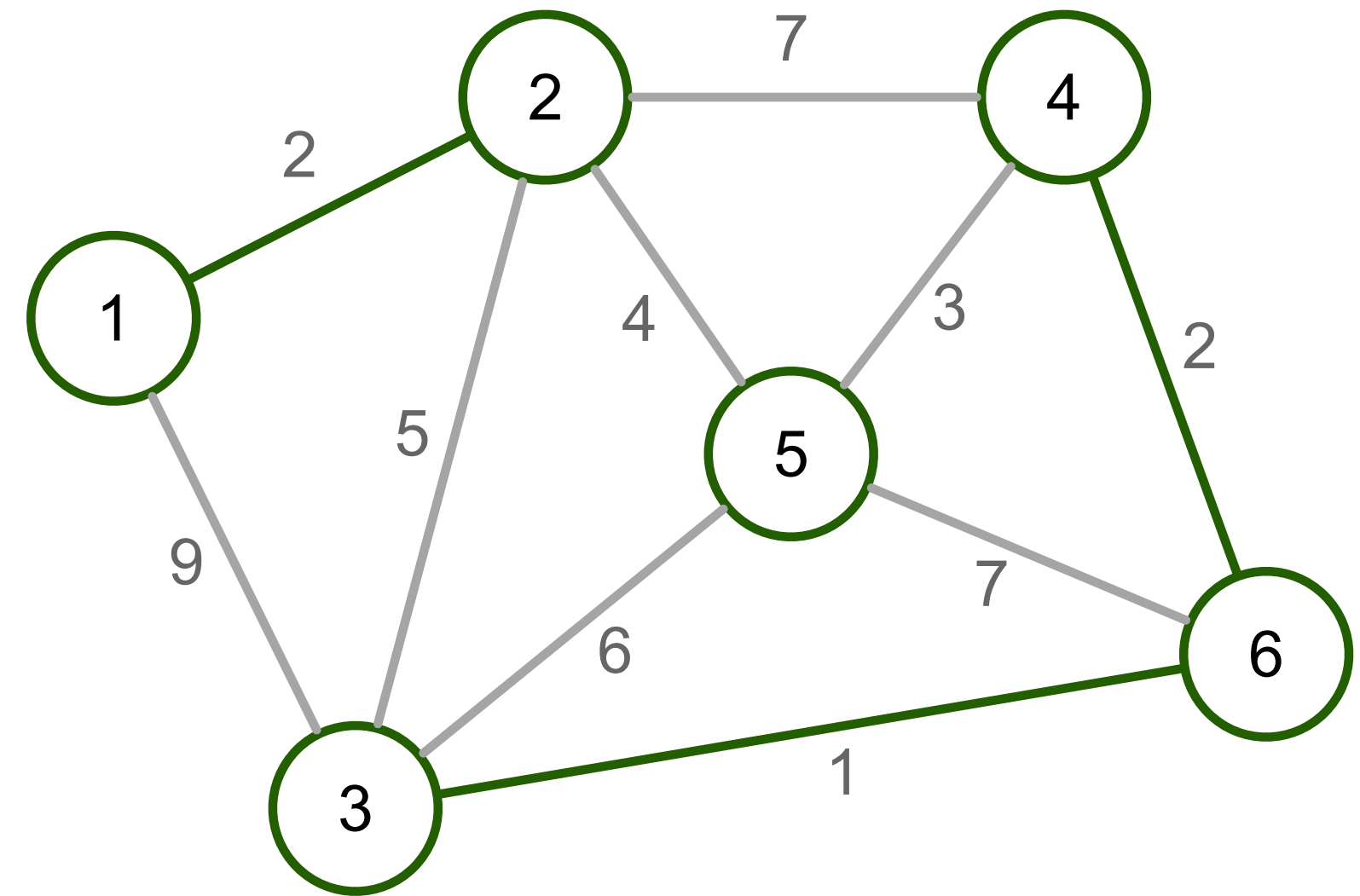
- O algoritmo de **Kruskal** foi publicado por Joseph Kruskal em 1956.
- Esse algoritmo é capaz de encontrar a MST de um grafo $G = (V, E)$.
- A estratégia desse algoritmo consiste em crescer uma floresta $F = (V', E')$ até que ela se torne uma árvore geradora - $F = (V, E')$.



Árvore Geradora Mínima

Algoritmo de Kruskal

- Uma aresta e_k é externa à floresta F se $e_k \notin F$ e o grafo $F + e_k$ é uma floresta.
- Ideia geral do algoritmo:
 - Inicialize a floresta F com todos os vértices de G e nenhuma aresta.
 - Escolha a aresta $e_k = (v_i, v_j)$ de G que possua o menor custo.
 - Insira e_k em F .



Árvore Geradora Mínima

Algoritmo de Kruskal

- A seguir uma implementação simplória (e ineficiente) do algoritmo:

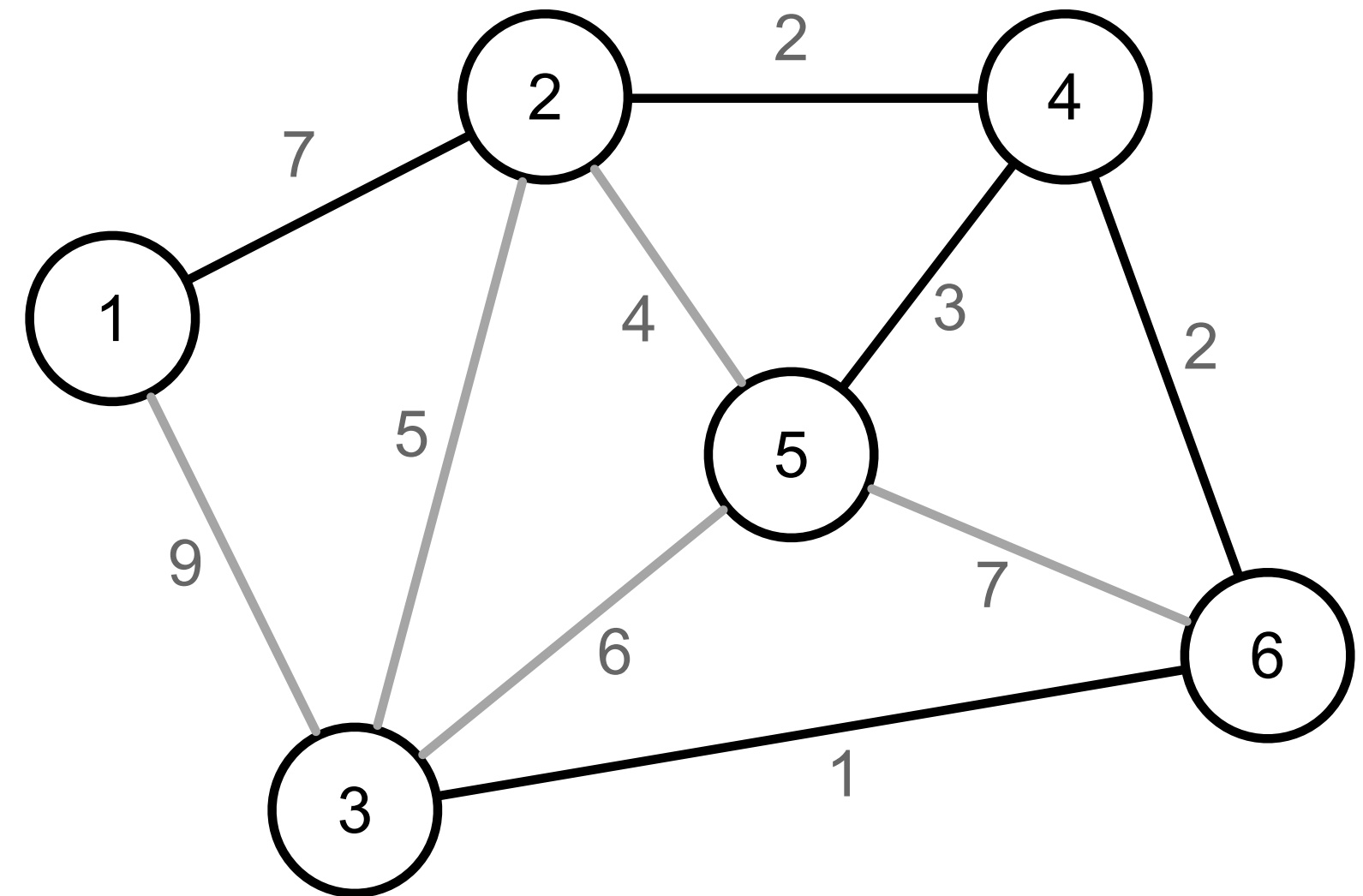
```
void mstKruskalSlow (Edge * edges) {
    vertex group[m_numVertices];
    for (vertex v=0; v < m_numVertices; v++) { group[v] = v; }
    int k = 0;
    while (true) {
        int minCost = INT_MAX;
        vertex minV1, minV2 = -1;
        for (vertex v1=0; v1 < m_numVertices; v1++) {
            EdgeNode * edge = m_edges[v1];
            while (edge) {
                vertex v2 = edge->otherVertex();
                int cost = edge->cost();
                if (v1 < v2 && group[v1] != group[v2] && cost < minCost) {
                    minCost = cost;
                    minV1 = v1;
                    minV2 = v2;
                }
                edge = edge->next();
            }
        }
        if (minCost == INT_MAX) return;
        edges[k++] = Edge(minV1, minV2, minCost);
        vertex leaderV1 = group[minV1];
        vertex leaderV2 = group[minV2];
        for (vertex v=0; v < m_numVertices; v++) {
            if (group[v] == leaderV2) {
                group[v] = leaderV1;
            }
        }
    }
}
```

- Analise da complexidade:
 - O loop mais externo adiciona uma aresta por vez, até encontrar a árvore geradora
 - A cada iteração procura pela aresta com menor peso e a adiciona na floresta
 - O dessa operação é $\sum_{i=1}^{|V|} g_s(v_i) = |E|$.
 - Em seguida atualiza os grupos iterando por um vetor de tamanho V .
- Portanto, a complexidade desse algoritmo é $O(V(E + V))$ no pior caso.

Árvore Geradora Mínima

Algoritmo de Kruskal

- A corretude do algoritmo pode ser avaliada através do **critério de minimalidade baseado em ciclos**.
- Uma árvore geradora T é uma MST de G se e somente se cada aresta $e_k \notin T$ apresentar o maior custo no ciclo fundamental de e_k relativo à T .



Árvore Geradora Mínima

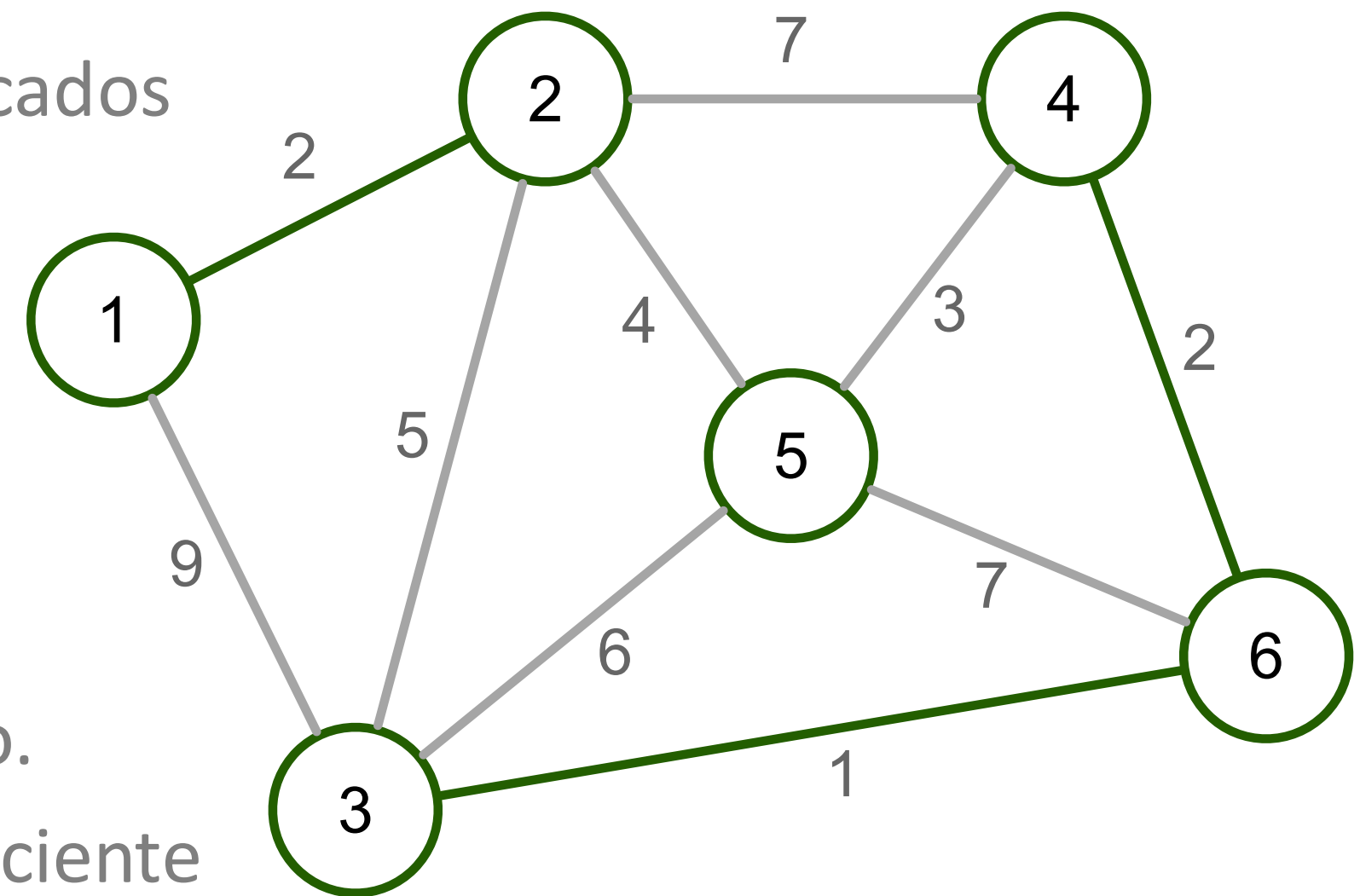
Algoritmo de Kruskal

- A versão simplória do algoritmo é lenta por dois motivos:

- A cada iteração todas as arestas são verificadas em busca da com menor custo.
- A cada iteração todos os vértices são verificados para avaliar se é necessário atualizar seu grupo.

- Podemos encontrar uma solução mais eficiente:

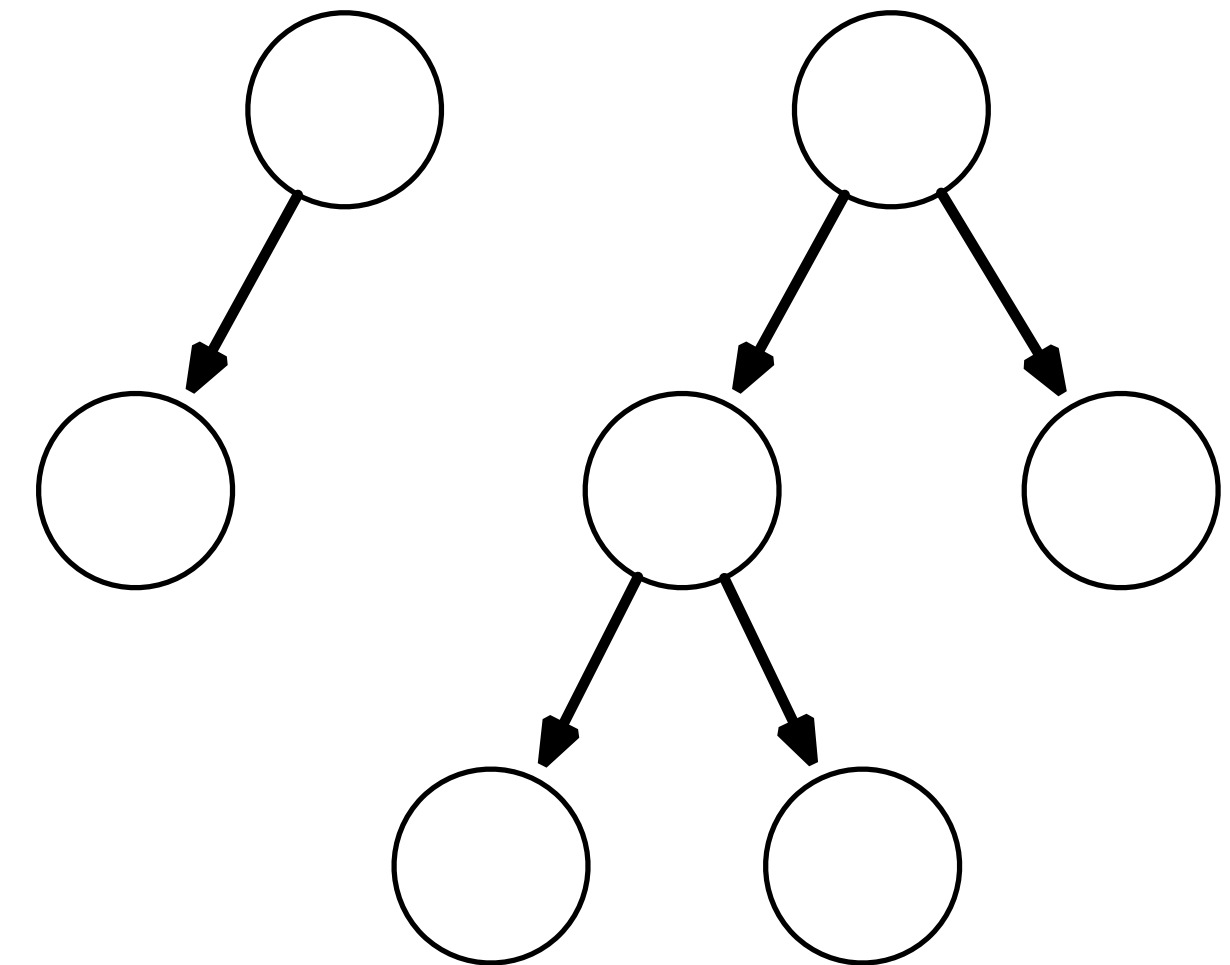
- Ordenando a lista de arestas pelo seu custo.
- Utilizando uma estrutura de dados mais eficiente para fazer a busca e união dos vértices.



Árvore Geradora Mínima

Algoritmo de Kruskal

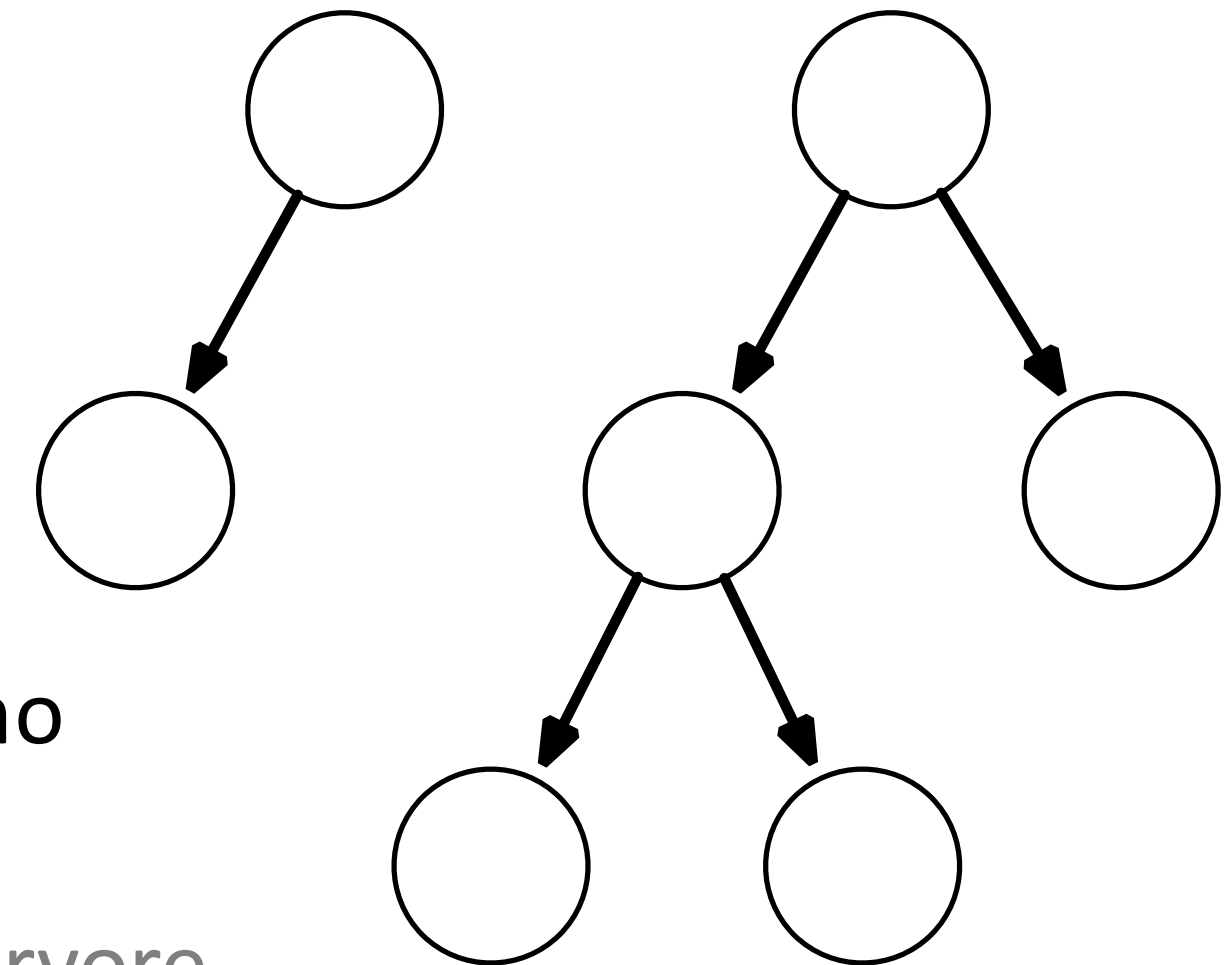
- A estrutura **union-find** facilita a manipulação de conjuntos por reduzir a complexidade nas operações de pesquisa e união.
- Todo elemento é associado à um conjunto
 - $group[v] = v$, se for o líder do grupo.
 - $group[v] = v'$, se não for o líder do grupo.
- Essa representação força uma estrutura de árvore entre os elementos de um mesmo conjunto.
- Também é armazenado o tamanho de cada conjunto.



Árvore Geradora Mínima

Algoritmo de Kruskal

- Seguindo essa abordagem podemos comparar se dois elementos pertencem ao mesmo grupo verificando se o líder de cada grupo é o mesmo.
- A operação de pesquisa consiste em obter o pai de cada elemento até chegar na raiz - o líder
 - Essa operação consome $O(\log(n))$ no pior caso.
- A união de grupos passa a ser realizada definindo como pai do menor conjunto o pai do maior conjunto
 - Essa abordagem reduz o crescimento da altura da árvore.
 - A operação é executada em tempo constante.



Árvore Geradora Mínima

Algoritmo de Kruskal

■ Exemplo de implementação de union-find:

```
class UnionFind {
public:
    UnionFind(int numElements)
    : m_numElements(numElements)
    , m_group(nullptr)
    , m_groupSize(nullptr) {
        m_group = new int[numElements];
        m_groupSize = new int[numElements];
        for (int e=0; e < numElements; e++) {
            m_group[e] = e;
            m_groupSize[e] = 1;
        }
    }

    int findE(int e) {
        vertex eLeader = e;
        while (eLeader != m_group[eLeader])
            eLeader = m_group[eLeader];
        return eLeader;
    }
}
```

```
void unionE(int e1, int e2) {
    if (m_groupSize[e1] < m_groupSize[e2]) {
        m_group[e1] = e2;
        m_groupSize[e2] += m_groupSize[e1];
    } else {
        m_group[e2] = e1;
        m_groupSize[e1] += m_groupSize[e2];
    }
}

private:
    int m_numElements;
    int * m_group;
    int * m_groupSize;
};
```

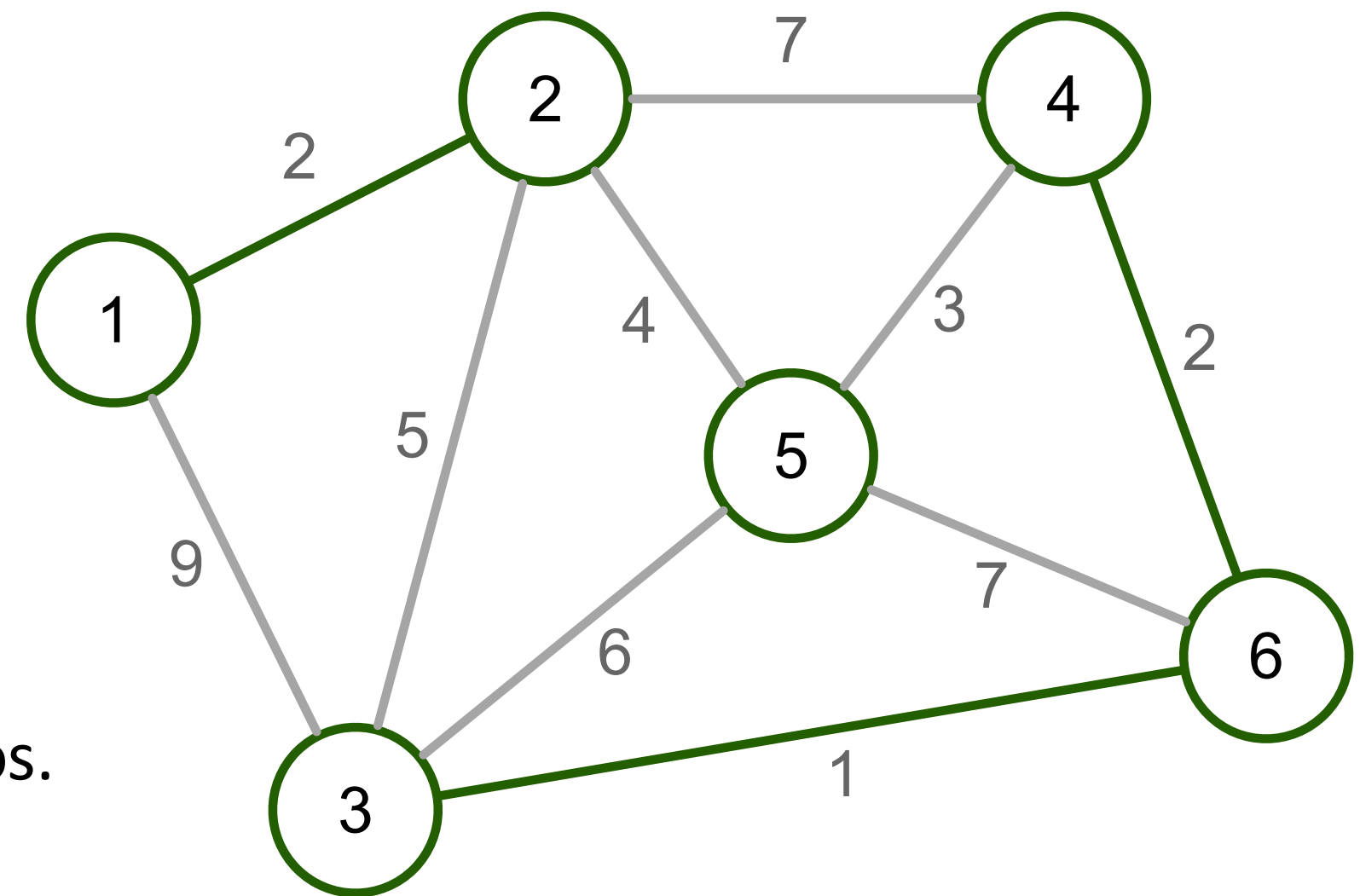
Árvore Geradora Mínima

Algoritmo de Kruskal

- Voltando ao algoritmo de Kruskal, podemos otimizá-lo utilizando union-find.

- Ideia geral do algoritmo:

- Inicialize a floresta F com todos os vértices de G e nenhuma aresta.
- Crie uma lista com todas as arestas e ordene pelo custo.
- Para cada aresta $e_k = (v_i, v_j)$
 - Obtenha os líderes dos vértices v_i e v_j .
 - Se forem diferentes, efetue a união dos grupos.
 - Insira e_k em F .
 - Pare ao encontrar $V - 1$ arestas.



Árvore Geradora Mínima

Algoritmo de Kruskal

- Uma possível implementação desse algoritmo seria:

```
void mstKruskalFast(Edge * mstEdges) {
    vector<Edge> edges(m_numEdges);
    int currentEdge = 0;
    for (vertex v1=0; v1 < m_numVertices; v1++) {
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            if (v1 < v2) {
                edges[currentEdge++] = Edge(v1, v2, edge->cost());
            }
            edge = edge->next();
        }
    }
    sort(edges.begin(), edges.end(), compareEdges);
    UnionFind uf(m_numVertices);
    currentEdge = 0;
    for (int e=0; currentEdge < m_numVertices - 1; e++) {
        Edge & edge = edges[e];
        vertex leaderV1 = uf.findE(edge.v1());
        vertex leaderV2 = uf.findE(edge.v2());
        if (leaderV1 != leaderV2) {
            uf.unionE(leaderV1, leaderV2);
            mstEdges[currentEdge++] = edge;
        }
    }
}
```

- Analise da complexidade:
 - A lista de arestas é ordenada em $O(E \log(E))$.
 - O loop seguinte adiciona uma aresta por vez, até encontrar a árvore geradora
 - A cada iteração obtém o líder de cada vértice e executa a união em $O(\log(V))$.
- Portanto, a complexidade é $O(E \log(E) + V \log(V))$
 - Como $\log E < 2 \log V$, temos $O((E + V) \log(V))$.

Perguntas?

`thiago.p.araujo@fgv.br`